Ander Urien Telleria

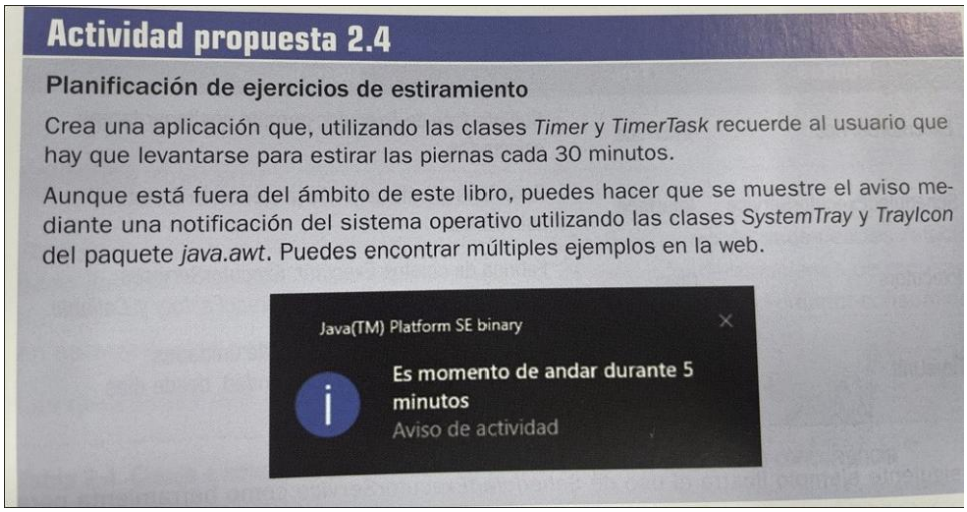
TIMER / TIMER TASK ariketa

Zerbitzu eta Prozesuen Programazioa (0490) • Timer eta TimerTask

> • Modulua: 0490 - Zerbitzu eta Prozesuen Programazioa (2. maila)
> • Gaia: Hariak (Threads), Aldizkako Atazak eta Baliabideen Kudeaketa

1. Sarrera eta Ariketaren Deskribapena

Zerbitzu eta Prozesuen Programazioan (ZPP), funtsezko gaitasunetako bat da aplikazioak atzeko planoan zereginak noiz eta nola exekutatu behar dituen programatzea, erabiltzailearen interfazeari edo hari nagusiari eragin gabe.



1. Irudia: Liburuko proposatutako ariketa eta sistemaren jakinarazpen-oharra.

2. Kontzeptu Teorikoak eta Erabilitako Klaseak

2.1. java.util.TimerTask

TimerTask klase abstraktu bat da, Java-ko Runnable interfazea inplementatzen duena. Klase honen helburua da Timer batek aldizka exekutatu beharreko lana kapsulatzea.

- **public abstract void run():** Ataza exekutatzen den bakoitzean metodo honi deituko zaio automatikoki. Bertan kokatzen da 30 minuturo burutu beharreko logika.
- **public boolean cancel():** Ataza bertan behera uzteko erabiltzen da.

2.2. java.util.Timer

Timer klaseak atzeko planoko hari independente bat kudeatzen du. Hari horren ardura da programatutako atazak une egokian abiaraztea.

- **scheduleAtFixedRate(TimerTask task, long delay, long period):** Maiztasun erregular zehatza bermatzen du. Ataza baten exekuzioak apur bat atzeratuko balu ere, hurrengo deiak uneko ordutegi absolutura doitzen ditu.
- **cancel() eta purge():** Timer haria amaitzeko eta ilaran dauden baliabideak garbitzeko ezinbesteko metodoak dira.

2.3. java.awt.SystemTray eta TrayIcon

Mahaigaineko sistema eragilean jakinarazpen natiboak bistaratzeko erabiltzen dira. Aplikazioak SystemTray.isSupported() bidez egiaztatzen du sistemak funtzio hau onartzen duen, ingurune headless-etan erroreak saihesteko.

3. Kodearen Azalpena

3.1. LuzatzeAtaza.java: Atazaren eraikuntza

Atazak TimerTask heredatzen du eta eraikitzailean TrayIcon jasotzen du sistemako abisuak bidali ahal izateko:

LuzatzeAtaza.java

```java
package org.example;

import java.awt.TrayIcon;
import java.time.LocalTime;
import java.time.format.DateTimeFormatter;
import java.util.TimerTask;

/**
 * LuzatzeAtaza klasea:
 * TimerTask hedatzen du. 30 minuturo exekutatuko den abisua definitzen du.
 */
public class LuzatzeAtaza extends TimerTask {  2 usages

    private final TrayIcon trayIcon;  3 usages
    private int abisuKontagailua = 0;  3 usages
    private static final DateTimeFormatter FORMATUA = DateTimeFormatter.ofPattern("HH:mm:ss");  1 usage

    public LuzatzeAtaza(TrayIcon trayIcon) {  1 usage
        this.trayIcon = trayIcon;
    }
```

run() metodoan, uneko ordua kalkulatu, kontsolan jakinarazpena idatzi eta Windows-eko globo-oharra erakusten da:

```java
    @Override
    public void run() {
        abisuKontagailua++;
        String unekoOrdua = LocalTime.now().format(FORMATUA);

        // 1. Kontsolan mezua inprimatu
        System.out.println("------------------------------------------------------");
        System.out.println("[" + unekoOrdua + "] " + abisuKontagailua + ". ABISUA:");
        System.out.println(">> Hankak luzatzeko garaia da(5 minutuz ibili)");
        System.out.println("------------------------------------------------------");

        // 2. Windows-eko jakinarazpen-eremuan globo-abisua atera
        if (trayIcon != null) {
            trayIcon.displayMessage(
                    caption: "Aviso de actividad",
                    text: "Es momento de andar durante 5 minutos",
                    TrayIcon.MessageType.INFO
            );
        }
    }

    public int getAbisuKontagailua() {  no usages
        return abisuKontagailua;
    }
}
```

3.2. Main.java: Denboralizazioa eta Kudeaketa

Denbora-tarteak definitzen dira (30 minutu = 1.800.000 ms) eta proba-modu azkar bat (10 segundo) egiaztapen azkarretarako:

```
public class Main {

    // Ariketaren tarte erreala: 30 minutu (milisegundotan)
    private static final long TARTEA_30_MINUTU = 30L * 60 * 1000;  2 usages

    // Probetarako tartea: 10 segundo
    private static final long TARTEA_10_SEGUNDO = 10L * 1000;  1 usage

    // True = Proba azkarra (10 segundoro); False = 30 minuturo
    private static final boolean PROBA_MODUA = true;  1 usage
```

Timer-a sortu eta scheduleAtFixedRate bidez abiarazten da:

```
42
43          // 1. Windows ataza-barran ikonoa prestatu
44          TrayIcon trayIcon = sortuTrayIcon();
45
46          // 2. Timer objektua sortu
47          Timer timer = new Timer( name: "LuzatzeTimerHaria");
48
49          // 3. TimerTask ataza sortu
50          LuzatzeAtaza ataza = new LuzatzeAtaza(trayIcon);
51
52          // 4. Ataza programatu scheduleAtFixedRate bidez
53          timer.scheduleAtFixedRate(ataza, hasierakoItxaronaldia, errepikatzeTartea);
54
```

Kanpoko irudirik behar ez izateko, SystemTray ikonoa memorian sortzen da dinamikoki Graphics2D erabiliz:

```
75 @    private static TrayIcon sortuTrayIcon() {  1 usage
76          if (!SystemTray.isSupported()) {
77              System.out.println("[OHARRA] SystemTray ez dago eskuragarri ingurune honetan.");
78              return null;
79          }
80
81          try {
82              SystemTray tray = SystemTray.getSystemTray();
83
84              // Memorian 16x16 tamainako ikono urdin bat sortu 'i' letrarekin
85              BufferedImage image = new BufferedImage( width: 16,  height: 16, BufferedImage.TYPE_INT_ARGB);
86              Graphics2D g = image.createGraphics();
87              g.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);
88              g.setColor(new Color( r: 0,  g: 120,  b: 215));
89              g.fillOval( x: 1,  y: 1,  width: 14,  height: 14);
90              g.setColor(Color.WHITE);
91              g.setFont(new Font( name: "SansSerif", Font.BOLD,  size: 11));
92              g.drawString( str: "i",  x: 6,  y: 12);
93              g.dispose();
94
95              TrayIcon trayIcon = new TrayIcon(image,  tooltip: "Luzatze Ariketak");
96              trayIcon.setImageAutoSize(true);
97              tray.add(trayIcon);
98
99              return trayIcon;
100         } catch (Exception e) {
101             System.err.println("[ERROREA] Ezin izan da TrayIcon sortu: " + e.getMessage());
102             return null;
103         }
104     }
105 }
```

4. Ondorioak

- **Hari anitzeko arkitektura:** Timer-ak atzeko planoan sortzen duen hariari esker, aplikazio nagusiak ez du exekuzioa gelditzen eta erabiltzailearen sarrera itxaron dezake.
- **Baliabideak askatzea:** cancel() metodoa deitzea nahitaezkoa da, hariak memorian blokeatuta ez geratzeko.
- **Integrazio natiboa:** SystemTray eta TrayIcon erabiliz, erabiltzaileari mahaigaineko esperientzia osoa eskaintzen zaio, liburuko jardueraren eskakizun guztiak betez.